RAG: Baseline vs. Re-Ranking — Report

Name: Amitabh Anand
Roll No.: 2025121008
Repository: https://github.com/amitabh-anandpd/rag-biencoder-vs-crossencoder
Notebook: `rag_baseline_vs_rerank.ipynb`

Setup

Component	Choice
Document	GDPR, Regulation (EU) 2016/679, 88 pages
Chunking	800-character windows, 150 overlap, per page → 580 chunks
Bi-encoder	`sentence-transformers/all-MiniLM-L6-v2` (384-d), FAISS `IndexFlatIP` on normalised vectors (cosine)
Cross-encoder	`cross-encoder/ms-marco-MiniLM-L-6-v2`
LLM	`gemini-3.5-flash-lite` , temperature 0, same prompt for both pipelines
Pipelines	P1: bi-encoder top 10 → top 3 to LLM. P2: same top 10 → cross-encoder re-sort → top 3 to LLM

---

1. Architecture Comparison

The bi-encoder processes the query and each document chunk separately. It converts each input into a 384-dimensional vector, and their relevance is calculated using cosine similarity. Since the vectors are normalised, FAISS's inner product gives the same result as cosine similarity. An important advantage is that the chunk vectors can be generated and stored before a query arrives, making retrieval fast. However, the model compresses an entire chunk into a single vector without directly comparing it with the query. As a result, it can prioritise chunks that share similar words even when they do not answer the question. Small but important differences, such as the word "not" or an exception to a rule, can easily be overlooked.

The cross-encoder takes the query and a chunk together as a single input. This allows each query token to interact with the chunk's tokens through self-attention, helping the model distinguish between statements that discuss the same topic but have different meanings. For example, it can better match a question about when a company is not required to notify affected people with the corresponding legal exception. Instead of producing separate vectors, it returns a relevance score for each query–chunk pair. These scores are useful for ranking candidates, rather than as absolute measures of relevance. The drawback is that each pair requires a separate model pass, so the computation cannot be completed in advance. Both models use six-layer MiniLM architectures, meaning the main difference in this experiment is how they process the inputs, rather than their size.

---

2. Qualitative & Rank Shift Analysis

To choose the failure queries, I screened about 20 candidate questions using only retrieval and re-ranking, and kept the ones where the chunk containing the deciding clause sat outside the baseline's top 3. I also kept one easy query as a control.

Rank shifts of the key chunk

#	Failure type	Query	Key chunk	Bi-encoder rank	Cross-encoder rank
1	Keyword trap	Can my employer monitor me at work?	562 (p.84, Art. 88)	#3	**#1**
2	Exception	Can I get my personal data deleted, and when can the company refuse?	99 (p.13, Recital 65)	#6	**#1**
3	Negation	When does a company not need to tell affected people about a breach?	378 (p.53, Art. 34(3))	#6	**#2**
4	Buried chunk	Must every company appoint a data protection officer?	392 (p.55, Art. 37(1)(c))	#7	**#1**
5	Control	Can I take my data to another service provider?	102 (p.13, Recital 68)	#2	#1

Answer 1 vs. Answer 2

#	Answer 1 (baseline)	Answer 2 (re-ranked)	Failure in Answer 1
1	Correct (Art. 88 chunk was at #3)	Correct, adds collective agreements (p.29)	None, but 2 of 3 chunks were distractors
2	Lists erasure grounds only, nothing on refusal	Lists all 5 refusal grounds	Omitted essential clause
3	"The context does not explicitly list circumstances where a company does not need to tell affected people"	Gives encryption + later-measures exceptions	Omitted essential clause
4	"May designate… shall where required by law"	Gives mandatory large-scale condition	Misstated rule
5	Correct	Correct	None (control)

1. The baseline's top two results, chunks 195 and 196, discussed the monitoring and enforcement activities of supervisory authorities rather than an employer monitoring employees. The word "monitoring" created a strong topical match, even though the passages addressed a different question. Because the Article 88 chunk was already third, both answers were correct, but two of the three baseline passages were irrelevant. The baseline did not always produce a wrong answer, but its retrieval quality could still be improved.
2. The baseline's top three results focused on when personal data can be erased (Article 17(1) and the beginning of Recital 65). They matched the deletion part of the question but not the refusal part, so Answer 1 left out the refusal conditions entirely. After re-ranking, the rest of Recital 65 reached first place and Answer 2 listed all five refusal grounds.
3. The baseline retrieved passages about reporting breaches to supervisory authorities (Article 33) and general breach recitals (85–87). These matched the topic but not the negation, so Answer 1 stated that the context did not list the exceptions. With Article 34(3) promoted to second, Answer 2 identified the encryption and subsequent-measures exceptions, but still missed the third exception because it fell outside the retrieved chunk.
4. The baseline retrieved passages about DPO tasks and the "may designate" provision, which made the appointment appear optional. Once Article 37(1)(c) moved to first, Answer 2 explained the mandatory condition for large-scale processing of special categories of data. It was still incomplete, since the conditions in Article 37(1)(a) and (b) were not among the final three chunks.

Cross-cutting observations

Across these queries, re-ranking improved the ordering of passages that were already retrieved, particularly when the question depended on negation, exceptions or a specific legal condition. In query 1, the bi-encoder scores were tightly grouped between 0.315 and 0.367, while the cross-encoder scores

ranged from about −5.8 to −11.2, separating the relevant passage much more clearly. The negative scores are not a problem because their purpose is to rank candidates, not to represent probabilities. I also observed that the LLM did not simply invent missing rules. With the instruction to answer only from the supplied context, it acknowledged missing information in the baseline answer to query 3. This reduced the risk of unsupported claims, but incomplete answers can still be misleading when they sound definitive, as seen in query 4. The control query (query 5) remained correct in both pipelines: chunk 102, from Recital 68, moved from second to first without making the answer worse.

Limitations observed

Re-ranking cannot recover information that the initial bi-encoder fails to retrieve among its candidates. For example, the relevant text of Article 17(3) remained outside the top three, while the conditions in Article 37(1)(a) and (b) were also missed. The answer to query 2 relied on the recital rather than the article itself, and query 4 remained incomplete. Chunking was another limitation: the 800-character window cut Article 34(3) before its third exception. These issues could be reduced by retrieving more candidates, splitting documents along article and paragraph boundaries, and combining dense retrieval with keyword-based BM25 search.

---

3. Latency Profiling

Pipeline	Retrieval	Re-rank	Retrieval total	LLM call
1 – Baseline	6.6 ms	–	6.6 ms	~1.56 s
2 – Re-ranking	6.6 ms	20.2 ms	26.8 ms	~1.76 s

I measured latency on the CPU after a warm-up run, using 20 runs for each of the five queries and averaging the results; the LLM time is the mean of the real Gemini calls. The re-ranking stage added 20.2 ms, increasing retrieval time from 6.6 ms to 26.8 ms, or roughly four times the baseline retrieval time. However, compared with the full request time of approximately 1.6 seconds, the additional cost was only around 1–1.3%. This suggests that re-ranking can improve answer quality with little noticeable increase in end-to-end latency. The difference between the measured LLM times is most likely due to API and network variation, rather than an inherent slowdown caused by re-ranking.

---

4. Scaling Question

In the experiment, cross-encoding all 580 chunks for a single query took 886 ms (about 45 times the 10-candidate re-rank), which works out to approximately 1.5 ms per query–chunk pair. If this cost scaled linearly to a corpus of one million chunks, scoring every chunk would take approximately 25 minutes for

a single query. This would be impractical for an interactive question-answering system, especially when multiple users are querying it.

The bi-encoder avoids this cost by computing document vectors once, offline. At query time, it only needs to encode the query and search the existing vector index. Approximate nearest-neighbour indexes, such as HNSW and IVF, can make this search efficient even for millions of vectors. The key difference is that document representations can be precomputed, whereas query–document relevance scores depend on the query and must be calculated when needed.

For this reason, a two-stage retrieval pipeline is more practical. The bi-encoder first retrieves a small set of potentially relevant candidates, and the cross-encoder carefully re-ranks them. In my experiment, re-ranking ten candidates added only about 20 ms while correcting several important retrieval errors. This provides a useful balance between retrieval speed and answer quality.